FastAPI and ML Integrated AI-Agent

1. Executive Summary & Architecture Overview

Why Connect AI Agents to ML Models?

- **Large Language Models (LLMs):** Exceptional at natural language understanding, extraction, and reasoning, but incapable of precise real-time predictive math on custom datasets.
- **Machine Learning Models:** Exceptional at numerical calculations, pattern recognition, and predictions (e.g., price estimation), but lack conversational capabilities.
- **FastAPI:** Acts as the high-performance REST API bridge connecting the LLM Agent to the ML Model.

```
+-------------------------------------------------------------------------
----------+
|                                SYSTEM ARCHITECTURE
|
+-------------------------------------------------------------------------
----------+

 [ 1. User Prompt ] ──> "Estimate cost for a 2000 sqft, 3-bed house..."
         |
         ▼
 [ 2. AI Agent (Streamlit + OpenAI) ] ──> Extracts parameters into JSON
         |
         ▼
 [ 3. REST API Call (FastAPI) ] ──> POST /predict-price
         |
         ▼
 [ 4. Predictive ML Model (Scikit-Learn) ] ──> Processes features &
returns $385,000
         |
         ▼
 [ 5. AI Agent Synthesis ] ──> Combines prediction with tailored financial
advice
```

---

2. Pre-Lecture Task (Flipped Classroom Model)

Before attending the live session, you must build and test the **AI Agent Front-End Shell** on your local machine.

Task Objectives

1. Set up a local Streamlit chat application.
2. Configure OpenAI Function Calling schema for property valuation.
3. Write a temporary "mock" function that simulates a backend ML model.
4. Verify that the AI Agent correctly asks for missing user inputs before triggering the tool.

---

3. Step-by-Step Setup Sequence

1. **Environment Preparation:** Terminal Setup. Open your terminal or command prompt and install the required dependencies:

```
pip install streamlit openai
```

2. **Configure API Authentication:** Environment Variables. Set your Groq API key in your terminal session, as the updated code utilizes the Groq API endpoint:

Mac / Linux:

```
export GROQ_API_KEY="your-actual-api-key-here"
```

Windows (CMD):

```
cmd
set GROQ_API_KEY="your-actual-api-key-here"
```

Windows (PowerShell):

```
$env:GROQ_API_KEY="your-actual-api-key-here"
```

3. **Create the Shell Script:** File Creation. Create a new file named `agent.py` in your working directory and paste the complete Python code provided in Section 4.
4. **Execute & Verify:** Local Server. Launch the Streamlit app:

```
streamlit run agent.py
```

Test the interaction by asking: "*Can you give me a valuation for a 2500 sqft home with 3 bedrooms, 2 bathrooms, 5 years old, in a location rated 8?*"

Here are the step-by-step instructions to get your Groq API key:

1. **Visit the Groq Console:** Go to console.groq.com in your web browser.
2. **Sign In or Create an Account:** Log in using your existing email, Google, or GitHub account. If you are new to Groq, follow the prompts to sign up.

3. **Navigate to API Keys:** Once you are logged in to the dashboard, click on **API Keys** in the menu (or navigate directly to `[console.groq.com/keys](https://console.groq.com/keys)`).
4. **Create a New Key:** Click the **"Create API Key"** button. You will be prompted to give your key a descriptive name so you can easily identify it later.
5. **Copy and Store Securely:** Your new API key will appear on the screen. **Copy it immediately and save it somewhere secure**. For security purposes, Groq will only display this key to you once. If you lose it, you will need to generate a new one.
6. **Add to Your Environment:** Paste the key into your project's `.env` file or export it directly in your terminal as an environment variable (e.g., `export GROQ_API_KEY="your-api-key-here"`).

---

4. Complete Code Base (`agent.py`)

Note: This code has been updated to route through the Groq API using the `qwen/qwen3.8-27b` model.

```
import streamlit as st
import json
import os
from openai import OpenAI

# Initialize OpenAI client using the environment variable
#client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))

client = OpenAI(
    api_key=os.environ.get("GROQ_API_KEY"),
    base_url="[https://api.groq.com/openai/v1]
(https://api.groq.com/openai/v1)"
)

# Fetch and print all accessible model IDs for your account
#print("Models available to your API key:")
#for model in client.models.list().data:
#    print(f"- {model.id}")

st.set_page_config(page_title="Vi4 Academy AI Agent", layout="centered")
st.title("🏠 Vi4 Academy: Real Estate AI Agent")
st.markdown("Conversational AI powered by Function Calling & ML
integration.")

# ---------------------------------------------------------------------
------
# 1. TEMPORARY MOCK FUNCTION
# (During the live lecture, this will be replaced with an HTTP request to
FastAPI)
# ---------------------------------------------------------------------
```

```
------
def get_house_price_estimate(sqft, bedrooms, bathrooms, age_of_house,
location_score):
    """
    Placeholder logic to simulate an ML model response.
    """
    base_price = 100000
    price = base_price + (sqft * 150) + (bedrooms * 10000) + (bathrooms *
5000)
    price -= (age_of_house * 1000)
    price += (location_score * 15000)
    return {"estimated_price": price}

# ---------------------------------------------------------------------
------
# 2. OPENAI TOOL DEFINITION (JSON SCHEMA)
# ---------------------------------------------------------------------
------
tools = [
    {
        "type": "function",
        "function": {
            "name": "get_house_price_estimate",
            "description": "Estimate a property price based on house
features. Call this when a user requests a valuation.",
            "parameters": {
                "type": "object",
                "properties": {
                    "sqft": {"type": "integer", "description": "Total
interior area in square feet"},
                    "bedrooms": {"type": "integer", "description": "Total
number of bedrooms"},
                    "bathrooms": {"type": "integer", "description": "Total
number of bathrooms"},
                    "age_of_house": {"type": "integer", "description":
"Age of the house in years"},
                    "location_score": {"type": "integer", "description":
"Location rating scale from 1 (poor) to 10 (prime)"}
                },
                "required": ["sqft", "bedrooms", "bathrooms",
"age_of_house", "location_score"]
            }
        }
    }
]

# ---------------------------------------------------------------------
------
# 3. SESSION STATE & SYSTEM PROMPT
# ---------------------------------------------------------------------
------
if "messages" not in st.session_state:
```

```
    st.session_state.messages = [
        {
            "role": "system",
            "content": (
                "You are an expert Real Estate & Financial Advisor AI. "
                "When users ask for a price estimate, ensure you have all
required details: "
                "sqft, bedrooms, bathrooms, age_of_house, and
location_score (1-10). "
                "If any detail is missing, ask follow-up questions
politely. "
                "Once all parameters are gathered, execute the
get_house_price_estimate tool "
                "and provide strategic financial advice based on the
calculated valuation."
            )
        }
    ]

# Render chat history
for msg in st.session_state.messages:
    if msg["role"] not in ["system", "tool"]:
        with st.chat_message(msg["role"]):
            st.markdown(msg.get("content", ""))

# ---------------------------------------------------------------------
------
# 4. CHAT INTERACTION & REASONING LOOP
# ---------------------------------------------------------------------
------
if prompt := st.chat_input("Ask about home valuations or financial
advice..."):
    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)

    # Initial API call to evaluate user prompt and check for function
calls
    response = client.chat.completions.create(
        model="qwen/qwen3.8-27b", # <--- Changed from gpt-4o
        messages=st.session_state.messages,
        tools=tools,
        tool_choice="auto"
    )

    response_msg = response.choices[0].message

    # Store assistant payload
    msg_dict = {"role": "assistant", "content": response_msg.content}
    if response_msg.tool_calls:
        msg_dict["tool_calls"] = [
            {
```

```
                "id": t.id,
                "type": "function",
                "function": {"name": t.function.name, "arguments":
t.function.arguments}
            } for t in response_msg.tool_calls
        ]
    st.session_state.messages.append(msg_dict)

    # Execute Tool logic if triggered by LLM
    if response_msg.tool_calls:
        for tool_call in response_msg.tool_calls:
            if tool_call.function.name == "get_house_price_estimate":
                args = json.loads(tool_call.function.arguments)

                # Call local mock function
                result = get_house_price_estimate(
                    sqft=args.get("sqft"),
                    bedrooms=args.get("bedrooms"),
                    bathrooms=args.get("bathrooms"),
                    age_of_house=args.get("age_of_house"),
                    location_score=args.get("location_score")
                )

                # Append tool output to chat context
                st.session_state.messages.append({
                    "role": "tool",
                    "tool_call_id": tool_call.id,
                    "name": "get_house_price_estimate",
                    "content": json.dumps(result)
                })
        for msg in st.session_state.messages:
             print(msg)
        # Second API call to synthesize natural language advice using tool
data
        final_response = client.chat.completions.create(
            model="qwen/qwen3.8-27b",
            messages=st.session_state.messages
        )
        final_content = final_response.choices[0].message.content
        st.session_state.messages.append({"role": "assistant", "content":
final_content})

        with st.chat_message("assistant"):
            st.markdown(final_content)

    elif response_msg.content:
        with st.chat_message("assistant"):
            st.markdown(response_msg.content)
```

---

5. Code Concepts Breakdown

Component	Code Section	Purpose
Mock Tool	`def get_house_price_estimate()`	Simulates a remote predictive backend during testing.
Tool Schema	`tools = [...]`	Teaches the LLM what parameters it must extract from raw text.
System Prompt	`st.session_state.messages`	Directs the Agent's personality, requirement gathering, and advisory role.
Two-Step Loop	`response_msg.tool_calls`	Intercepts function execution requests from OpenAI, executes local code, and passes results back for final natural language generation.

---

6. Live Session Roadmap (What We Will Build in Class)

During the upcoming 3-hour session, we will complete the end-to-end integration:

Phase	Duration	Topic
Phase 1	45 Mins	**ML Model Training:** Train a Scikit-Learn Regression model on real estate data and export it via `joblib`.
Phase 2	60 Mins	**FastAPI Deployment:** Wrap the trained `.joblib` model inside a FastAPI app exposing a `/predict-price` POST endpoint with Pydantic data validation.
Phase 3	45 Mins	**Agent Tool Integration:** Replace the mock function in `agent.py` with an HTTP `requests.post()` call pointing to `http://localhost:8000/predict-price`.
Phase 4	30 Mins	**Testing & Q&A:** Full end-to-end system testing and edge-case handling.

---

www.Vi4Academy.com